



Improving LLMs' Reasoning and Planning with Finite-State Machines

Eduardo Faria Cabrera^(✉), Marcel Rodrigues de Barros,
and Anna Helena Reali Costa

Universidade de São Paulo, São Paulo, SP, Brazil
{eduardofcabrera,marcel.barros,anna.reali}@usp.br

Abstract. Large Language Models (LLMs) have shown remarkable capabilities across various applications but struggle with sequential decision-making tasks like planning. This work demonstrates that integrating LLMs with Finite-State Machines (FSMs) can enhance their reasoning and planning abilities, while also offering increased reliability. Several setup variants are compared, providing a better understanding of how validations, feedback loops, and restrictions enhance robustness and effectiveness. A comparison to the well known Chain-of-Thoughts approach is also provided. Our methods improve planning capabilities of all analysed LLMs, consistently increasing the success rate in solving tasks of varying complexities. A detailed analysis of the two best variants are provided, highlighting their respective strengths and weaknesses.

Keywords: Large Language Models · Machine Learning · Planning · Graph Representations

1 Introduction

Recently, Large Language Models (LLMs) have surged in development, revolutionizing AI capabilities. Scaling up these models has unlocked new feats, from crafting narratives and summarizing text to coding and enhancing reasoning skills [2, 3, 18]. Despite these advancements, LLMs still face challenges in basic logical reasoning, often producing implausible claims and struggling with rudimentary planning tasks [2, 15]. Moreover, the opacity of their decision-making processes poses significant obstacles in interpreting and validating their outputs, especially in critical domains like healthcare and finance. Thus, there is an urgent need for methodologies to validate, guide, and interpret the decision pathways of LLMs.

One extensively studied form of reasoning task in the AI community is planning and making sequential decisions. Planning involves executing a series of

The authors thank the Centro de Ciência de Dados (C2D-Poli-USP) supported by Itaú Unibanco S.A., as well as the Center for Artificial Intelligence (C4AI-USP), supported by FAPESP (grant 2019/07665-4) and IBM Corporation. This work is partially supported by CNPq (grant number 312360/2023-1) and CAPES (Finance Code 001).

valid actions within a defined context to transition from an initial state to a goal state [11]. Even simple planning tasks pose challenges for LLMs, as they often struggle to achieve success rates and generate valid plans due to the requirement of validating each action’s feasibility in every state, backtracking previous states and actions, and employing long-term reasoning [15]. Enhancing planning, reasoning, and introducing guarantees in multi-step decision making problems is crucial for improving LLMs’ reliability, which would open up their usage in some risk averse applications.

The research community is actively developing new methods to address multi-step problems with LLMs by enhancing their reliability, interpretability, and robustness. One such technique is Chain-of-Thought (CoT) [19], a prompt-based approach that incorporates intermediate reasoning steps, or “thoughts,” alongside task input/output pairs. By decomposing the original problem into these intermediate steps, CoT facilitates structured reasoning, thereby improving LLMs’ ability to tackle procedural problems. Empirical evidence has demonstrated CoT’s effectiveness in enhancing problem-solving capabilities [19]. Furthermore, advancements have extended CoT’s capabilities, including techniques like Self-Consistency with CoT (CoT-SC) [17] and CausalCOT [9], as well as more complex and flexible approaches like Tree-of-Thoughts [20] and Graph-of-Thoughts [1]. However, these techniques primarily focus on refining prompting strategies without explicitly leveraging the logical structure of problems to validate and guide LLMs’ reasoning processes.

Our approach associates LLMs to Finite-State Machines (FSMs), commonly used in systems that require reliability and predictability, as a guiding structure to the decision-making processes of LLMs. This association restricts the model’s decision at each step, providing several additional guarantees enabling applications with tighter restrictions to benefit from modern AI capabilities.

The key contributions of our work are:

Novel Approach: We introduce a novel method that incorporates FSMs as a guiding structure for LLM decision-making processes.

Enhanced Problem-Solving: We provide evidence that validations, feedback loops, and restrictions enabled by FSMs, derived from standard planning language definitions, enhance LLMs’ ability to solve planning problems.

Comprehensive Comparison: We present a detailed comparison of our most effective setups, thoroughly analyzing their strengths and weaknesses in the context of planning problems.

2 Background

This section covers essential concepts for this work. It briefly explains how autoregressive LLMs function and their advantages in solving planning problems. It then provides a formal definition of a planning problem. Finally, it defines a Mealy Machine, a type of Finite-State Machine, which will be used to guide the LLM in addressing planning problems.

2.1 Large Language Models for Planning

Large language models are built upon a transformer architecture [16], which undergoes training on a vast corpus of text samples sourced from the web. Formally, we define a collection of examples $X = (x_1, x_2, \dots, x_N)$, where each $x_i = (\alpha_1, \alpha_2, \dots, \alpha_{n_i})$ and every token α_i is drawn from a predefined vocabulary $\mathbf{V}$. Each LLM is equipped with a tokenizer responsible for converting raw input text into a sequence of tokens, each belonging to $\mathbf{V}$. LLMs typically undergo training in an auto-regressive manner, where the probability of each token is solely dependent on its preceding tokens in the sequence, expressed as $p(x_i) = \prod_{j=1}^{n_i} p(\alpha_j | \alpha_{0:j-1})$. During training, the model parameters are learned by maximizing the likelihood of the entire dataset, formulated as $p(X) = \prod_{i=1}^N p(x_i)$.

Addressing planning problems with pre-trained LLMs is highly advantageous as these models can leverage their extensive world knowledge, commonsense understanding, and logical reasoning abilities. The aim is to generate a sequence of actions $(a_1, a_2, \dots, a_n)$ to reach a desired final state. Graph structures have been widely used to enhance the reasoning capabilities of LLMs, with empirical evidence showing significant improvements in solving complex problems [13, 17, 19]. The CoT approach is fundamental, modeling the inference process as a linear chain and explicitly capturing intermediate reasoning steps. Building on CoT, methods like Tree-of-Thoughts and Graph-of-Thoughts introduce more complex topologies, including branches, backtracking, and aggregating reasoning steps.

Nonetheless, incorporating prompt topologies alone is insufficient to tackle planning problems. Studies such as [15] underscore the difficulties LLMs face in generating valid and optimal plans. Recent research, exemplified in [10, 12], suggests that integrating feedback loops can significantly improve LLMs' accuracy in planning tasks. Consequently, implementing a mechanism for validating the model's outputs and providing feedback emerges as a promising strategy for enhancing its performance.

2.2 Planning Task Definition and Finite-State Machines

A planning problem can be defined as a tuple $\mathcal{P} = (\mathcal{S}, \mathcal{A}, s_0, \mathcal{G})$ [11], where:

- $\mathcal{S}$ is a finite set of states. Each state $s \in \mathcal{S}$ is a complete assignment of values to a set of state variables (or fluents).
- $s_0 \in \mathcal{S}$ is the initial state.
- $\mathcal{G} \subset \mathcal{S}$ is the set of goal states where the goal condition is satisfied. This condition is specified as a conjunction of literals.
- $\mathcal{A}$ is the set of actions, where each action $a \in \mathcal{A}$ is defined by its preconditions and effects:
 - Preconditions $pre(a)$ are a set of literals that must be satisfied for the action to be applied.
 - Effects $eff(a)$ are a set of literals that describe how the state changes after applying the action.

We define a transition model $\gamma : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ that describes deterministic state transitions in the planning problem. If a state $s \in \mathcal{S}$ satisfies $\text{pre}(a)$, then $\gamma(s, a) = s'$, where $s' \in \mathcal{S}$ is the new state resulting from applying $\text{eff}(a)$ to s . If s does not satisfy $\text{pre}(a)$, then the action cannot be applied.

The Planning Domain Definition Language (PDDL) is commonly used to represent planning problems, allowing for the encoding of a problem by defining its domain (types of objects, predicates, and actions) and its problem instance (initial state and goal state). There is extensive literature on planning algorithms for various tasks. An AI planner takes a PDDL domain and problem as input and produces a solution. Most state-of-the-art planners [6, 7] are variants of heuristic-based forward search, where PDDL encodings are used to automatically derive a heuristic function.

Another common approach is the integration of Markov Decision Processes (MDPs) into automated planning. MDPs are best suited for modeling stochastic processes where maximizing rewards in uncertain contexts is crucial [4, 11]. However, many mission-critical applications are modeled using deterministic finite-state machines due to reliability and predictability requirements.

Building on this foundation, our work investigates whether simply imposing an alphabet of interactively validated symbols can enhance LLMs' ability to solve planning tasks. This restriction of possible actions associated with deterministic transitions is best encapsulated by the FSMs formalism.

A Mealy Machine is an FSM defined by a 6-tuple $(\mathcal{S}, s_0, \Sigma, \Lambda, \mathcal{T}, \mathcal{O})$, where:

- $\mathcal{S}$ is a finite set of states.
- $s_0 \in \mathcal{S}$ is the initial state.
- Σ is a finite set of input alphabet.
- Λ is a finite set of output alphabet.
- $\mathcal{T} : \mathcal{S} \times \Sigma \rightarrow \mathcal{S}$ is a transition function.
- $\mathcal{O} : \mathcal{S} \times \Sigma \rightarrow \Lambda$ is the output function.

3 Associating LLMs to FSMs

In this setup, we denote the LLM as a function $L(p) = t$, where a prompt $p \in \mathbf{V}^*$ is provided, and the output $t \in \mathbf{V}^*$ is a text response. Note that both input and output can be any sequence of tokens $(\alpha_1, \dots, \alpha_n) \in \mathbf{V}^*$, however, only a small fraction of this domain constitute valid actions.

The FSM can be derived from the problem's PDDL and is designed to receive an action $a \in \Sigma \subset \mathbf{V}^*$ and return an output $o \in \Lambda \subset \mathbf{V}^*$, which is appended to the next LLM prompt p . This establishes an interactive loop between the LLM and the FSM, where the language model proposes actions based on its context p , while the FSM models preconditions and effects. Note that in this setup there's no guarantee that the LLM output t is a valid action, which means the LLM can send invalid actions (i.e. symbols) to the FSM.

However, it is possible to project t onto Σ by providing a validation function f such that

$$f : (\mathbf{V}^*, \mathcal{S}) \rightarrow \Sigma \quad (1)$$

$$f(t_i, s_j) = \begin{cases} t_i, & t_i \text{ is a valid action in state } s_j \\ -1, & \text{otherwise} \end{cases} \quad (2)$$

This function checks whether t corresponds to a valid action for the current state, producing a common symbol -1 for invalid actions. Consequently, instead of the FSM’s transition and output functions receiving t , they use the output from the validation function: $\mathcal{T}(s_j, f(t_i, s_j))$ and $\mathcal{O}(s_j, f(t_i, s_j))$.

To assess how FSM modeling can enhance the capabilities of LLMs, we evaluate seven distinct methods. These range from a straightforward preplanned input/output setup to more sophisticated approaches that utilize FSMs to receive information and provide feedback on their actions. Additionally, we examine the CoT approach, as outlined in [15], where the initial prompt induces the model to generate structured reasoning based on a similar example.

By analysing these variations we aim to measure three key aspects:

1. Which method proves more effective: generating the entire plan at once, or generating each action individually while interacting with the FSM?
2. Does integrating validation functions and feedback loops improve LLMs’ effectiveness in solving planning tasks?
3. What methods can be employed to restrict the range of actions available to LLMs, thereby guiding them towards optimal outcomes?

Initially, all methods receive the same context prompt p_c sourced from [14]. This prompt includes a detailed description of the planning task, a rulebook outlining all predicates and actions, and information about the current and goal states, as shown in Fig. 1. Each setup has its own request prompt p_r , which solicits either the entire plan or individual actions, enabling customization based on each setup’s specific approach.

When the inference begins, the epsilon-transition $\mathcal{T}(s_{\text{start}}, \epsilon) = s_0$ occurs and the respective output $\mathcal{O}(s_{\text{start}}, \epsilon) = p_c + p_r$ produces the initial prompt. Thus, the state space becomes $\mathcal{S} = \{s_{\text{start}}, s_{\text{end}}\} \cup \mathcal{S}_{\text{task}}$, where $\mathcal{S}_{\text{task}}$ is the set of all possible states of the task being planned.

The methods analyzed are combinations of three characteristics:

Preplanned vs Interactive: Preplanned methods involve requesting the whole sequence of actions to the LLM before performing any kind of validation. This setup was assessed in [15], revealing a low success rate and a high prevalence of invalid plans. In this setup, the LLM generates the entire plan $(t_1, t_2, \dots, t_n)$ solely based on $p_c + p_r$. On the other hand, interactive methods require one action at a time, performing an interaction between FSM and LLM at each step, allowing for invalid actions to be recognized earlier, preventing a reset back to s_0 .

No Validation vs Validated vs Restricted: Three different schemes of validation are analyzed, namely, *no validation*, *validated*, and *restricted*. The *no validation* scheme simply halts the process in the case of invalid plans or actions. The *validated* scheme makes use of f to guarantee LLM outputs are projected onto Σ and provides feedback in the case of invalid actions. Finally, *restricted* methods provide all valid actions (obtained from the PDDL) for a certain state to the LLM through $\mathcal{O}$. In this case the LLM is asked to choose the number t_i corresponding to the selected action, but can still output any $t \in \mathbf{V}^*$. To handle invalid choices and to convert the selected action into a valid symbol we make use of a modified f

$$f(t_i, s_j) = \begin{cases} a_{t_i}^{s_j} & t_i \in p_a(s_j), \\ -1 & \text{otherwise.} \end{cases} \quad (3)$$

where $a_{t_i}^{s_j}$ is the action that corresponds to option t_i at state s_j .

Standard vs Chain-of-Thoughts: The standard prompting setup simply describes the goal and requests a plan (or the next action). On the other hand, CoT prompting induces the LLM to perform a step-by-step reasoning process for each interaction with the FSM. Prompting strategy can be modified by changing the prompts produced by $\mathcal{O}$.

Context Prompt	Preplanned - No Validation
Domain description: a brief summary of the domain. Rulebook: a detailed list of each predicate and action. Planning problem: descriptions of the current state and the goal state.	Context Prompt Return the plan that makes me achieve my goal: action₁ action₂ action_n } Generate full plan at once

Fig. 1. (Left) Context prompt used in all proposed methods. (Right) Preplanned - No Validation: We prompt the LLM to generate the complete plan in one step.

4 Experimental Setup

The dataset is obtained from [14], which provides a platform to insert a valid PDDL domain and generate instances of it. For this study, we utilized the Block-world domain, which captures common-sense block manipulations and consists of a set of blocks. Each block is uniquely identified by color and can be placed either on a table or on top of other blocks. In each instance, the initial state is a specific block arrangement, and the goal state is to arrange some of these blocks in a stack in a particular order (Fig. 2).

The Blockworld domain includes four different actions: *pick up*, *put down*, *stack*, and *unstack*. The preconditions for each action are as follows:

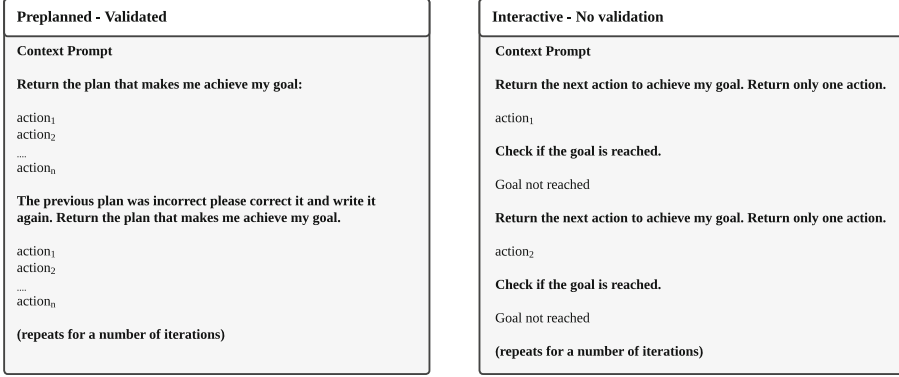


Fig. 2. (Left) Preplanned - Validated: We prompt the LLM to generate the entire plan in a single step. If the plan is invalid, we provide feedback and prompt the LLM to generate a new plan. (Right) Interactive - No Validation: We prompt the LLM to generate one action at a time and check if the goal is reached after each action. This process continues until the LLM indicates that the goal has been reached or the maximum number of iterations is exceeded.

- To *pick up* a block, it must be *clear* (no block stacked on it), *on the table*, and the hand must be *free*.
- To *put down* a block, it must be *on hand*.
- To *stack* a block on another, the first block must be *on hand* and the second block must be *clear*.
- To *unstack* a block from another, the first block must be *clear*, stacked on the second block, and the hand must be *free*.

This particular domain is widely adopted in the planning literature due to its simplicity and alignment with common-sense reasoning [5, 8].

In this work, we generated a dataset with 768 instances, each containing 4 blocks. The dataset is balanced in terms of optimal plan length, which ranges from 2 to 12 actions. We use the optimal plan length as an indicator of the instance’s complexity. For each instance, we use the same context prompt as in [14]. All prompts are zero-shot, except for methods using a CoT approach, where a one-shot approach is required to induce structured reasoning.

We evaluated three different LLMs in this work: Mixtral:8x22b, Llama3:70b, and GPT-4o. The temperature parameter is set to 0 for all models, ensuring that each experiment is deterministic. Our goal is to verify the effectiveness of the proposed methods independently of the specific LLM used. Additionally, a maximum of 8 tries for the Preplanned - Validated approach and 16 invalid actions for the Interactive approaches are enforced (Fig. 3).

The following metrics are calculated for each experiment:

- *Success Plan*: Indicates if the generated plan successfully reaches the goal without any errors.

Interactive - Validated	Interactive - Restricted
Context Prompt Return the next action to achieve my goal. Return only one action. action₁ Good Action! Return the next action to achieve my goal. Return only one action. action₂ This action is not possible due to the set of restrictions. Return the next action to achieve my goal. Return only one action. action₃ (repeats for a number of iterations)	Context Prompt Possible Actions: 1: action₁ 2: action₂ 3: action₃ Return the number of the best next action to achieve my goal: 2 Possible Actions: 1: action₄ 2: action₅ Return the number of the best next action to achieve my goal: 1 (repeats for a number of iterations)

Fig. 3. (Left) Interactive - Validated: We prompt the LLM to generate one action at a time and provide feedback on whether the action is valid or not. This process continues until the goal is reached or the maximum number of iterations is exceeded. (Right) Interactive - Restricted: We prompt the LLM to choose one of all possible actions at the current state. This process continues until the goal is reached or the maximum number of iterations is exceeded.

- *Valid Plan*: Indicates if the generated plan is valid, meaning it does not contain any invalid actions.
- *Optimal Plan*: Indicates if the generated plan is the optimal plan.
- *Relative Plan Length*: The total number of actions in the generated plan divided by the optimal plan length.
- *Tokens*: The number of tokens processed during inference, measured in thousands.
- *Distance to Initial State*: The number of actions from the initial state to the final state of the generated plan.
- *Distance to Goal State*: The number of actions from the final state of the generated plan to the goal state.

5 Results

Numerical results are presented in Table 1, displaying average values of accuracy, optimal plan percentage, number of processed tokens and relative plan length metrics for all methods and models. The validated and restricted interactive methods consistently outperform other setups in most cases. They achieve higher accuracy than their preplanned and unvalidated counterparts regardless of the model.

An important finding is that the CoT approach achieved good results exclusively with the GPT-4o model, which is specifically trained to handle CoT reasoning. In contrast, the Mixtral and Llama models performed worse with the CoT scheme in both the preplanned and the interactive validated methods. This provides evidence that FSM-based validation provides increased accuracy in a more generic sense, being independent of how the particular LLM was trained.

Table 1. Numerical results for all methods and models. The metrics include accuracy percentage, number of tokens processed, and relative plan length, calculated based on all generated plans. The optimal plan percentage is calculated only for the successful plans. These results represent the average values across the entire dataset. Std refers to Standard prompt and CoT to Chain of Thought prompt.

	Accuracy(%) $\uparrow$			Optimal Plan (%) $\uparrow$		
	GPT-4o	Llama3	Mixtral	GPT-4o	Llama3	Mixtral
Preplanned - No validation - Std	7.2	2.9	0.78	100.0	63.3	83.3
Preplanned - No validation - CoT	53.4	7.9	4.2	81.5	32.8	53.1
Preplanned - Validated - Std	12.9	6.4	2.4	85.9	38.8	22.2
Interactive - No validation - Std	1.8	9.1	11.1	78.6	85.7	62.4
Interactive - Validated - Std	44.4	33.5	21.9	73.6	40.1	58.3
Interactive - Validated - CoT	56.6	2.5	1.1	34.6	14.3	9.1
Interactive - Restricted - Std	50.5	27.2	17.7	75.8	73.2	49.3
	Tokens (thousands) $\downarrow$			Relative Plan Length $\downarrow$		
	GPT-4o	Llama3	Mixtral	GPT-4o	Llama3	Mixtral
Preplanned - No validation - Std	0.55	0.56	0.56	1.1	1.1	1.1
Preplanned - No validation - CoT	1.5	1.5	1.5	1.2	1.4	1.3
Preplanned - Validated - Std	4.8	5.1	5.4	1.5	1.4	1.1
Interactive - No validation - Std	10.4	4.4	3.9	2.1	1.2	1.0
Interactive - Validated - Std	10.9	12.5	13.8	2.2	2.4	2.4
Interactive - Validated - CoT	46.5	57.2	60.2	1.7	3.2	3.1
Interactive - Restricted - Std	16.7	21.0	22.7	1.5	2.0	2.4

The optimal plan percentage, which is calculated for successful plans, is higher for preplanned setups for two main reasons: First, since preplanned methods generate fewer successful plans, often for simpler tasks, they tend to show a higher optimal plan percentage on average. Second, interactive validated setups often generate longer successful plans, because these methods have a deterministic function to hard stop inference when the goal state is reached, instead of relying on the LLM to recognize it. This allows for longer plans to eventually achieve the goal state in situations that other methods fail.

A downside of using interactive methods is their increased computational cost. As also shown in Table 1, the interactive setup processes an order of magnitude more tokens than the preplanned setups. This is because interactive schemes involve a sequence of prompts where all previous history is used as input, causing the size of each prompt to increase with each iteration. Special cases are those with the CoT scheme, which significantly increases the number of processed tokens for both preplanned and interactive setups. It’s important to note that there is potential for prompt optimization in these schemes, which could make them faster and cheaper.

As displayed in Table 2 the preplanned and interactive methods without validation struggle with a low percentage of valid plans, as they are not robust against LLM hallucinations. In contrast, the interactive validated and restricted methods, by design, handle invalid inputs from the LLM, ensuring that all generated plans are valid. Once again, it is relevant to note that using CoT with Mixtral and Llama models frequently returned invalid actions, particularly in unfamiliar formats, demonstrating that such prompt engineering techniques can be sensitive to various factors and may work unpredictably across different models.

Table 2. Percentage of valid plans for methods that cannot handle invalid input. These results represent the average values across the entire dataset.

	Valid Plan (%) ↑		
	GPT-4o	Llama3	Mixtral
Preplanned - No validation - Std	13.7	15.5	12.9
Preplanned - Validated - Std	14.6	25.3	19.7
Preplanned - No validation - CoT	57.8	17.4	6.3
Interactive - No validation - Std	17.7	24.7	21.1

Considering the notably superior performance of the validated and restricted interactive methods compared to others, our analysis will primarily focus on these methods.

Figure 4 displays the average accuracy for each optimal plan length, serving as a complexity measure for each task in the dataset. As the optimal plan length increases, accuracy decreases, indicating that planning capabilities are sensitive to the complexity of the tasks. Furthermore, the two setups show different behaviors: the Interactive Validated setup demonstrates less sensitivity to optimal plan length, achieving a significant number of correct plans for more complex tasks, particularly with GPT-4o. In contrast, the Interactive Restricted setup shows more pronounced sensitivity, achieving higher accuracy for simpler tasks but experiencing a decrease in accuracy for more complex ones.

This behavior is further analysed in Figs. 5 and 6, which represent the distance from the final state of the generated plan to the goal and initial state, respectively. These metrics are only calculated for generated plans that didn’t achieve the goal. Figure 5 shows that the two methods have a similar behavior: when they produce plans that don’t achieve the goal state, these plans tend to finish far from the goal state. On average, the final state of the predicted plans is as far as the initial state of each task, indicating that the final state is not a good approximation of the goal state. However, in Fig. 6, one can see the distance from the final state of the produced plan to the initial state. This visualization shows that the distances for the Interactive Validated method are greater than the distances for the Interactive Restricted method. Thus, validated methods

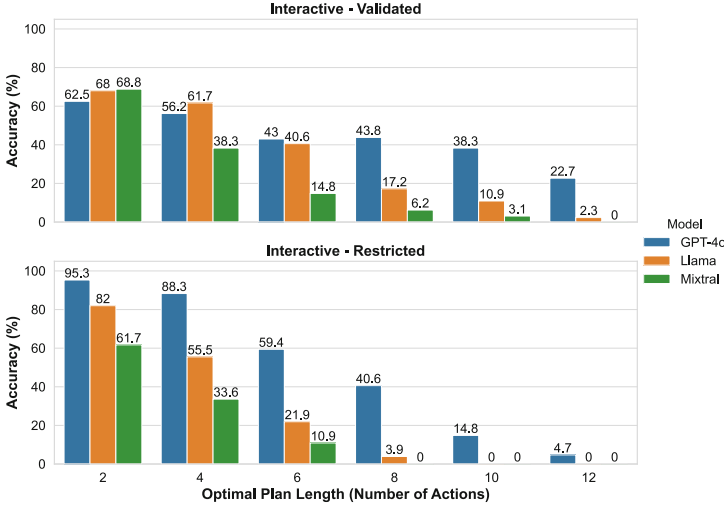


Fig. 4. The evaluation of average accuracy for the Interactive Validated and Restricted methods is divided by optimal plan length, which represents the minimal number of actions needed to achieve the goal state.

tends to explore more of the state space, resulting in final states farther away from the initial state, while restricted methods tend to perform local searches. This behavior is more pronounced for larger and more complex models, such as GPT-4o and Llama3.

Figure 7 presents further evidence for the difference in exploratory behavior between the methods. It shows the average number of times each model revisited states within a generated plan. We observe a significant difference: the Interactive Validated method exhibits a lower number of repeated states, indicating a tendency to explore diverse states. Conversely, the Interactive Restricted method frequently revisits states, suggesting a propensity to get stuck in loops. This looping behavior is likely inherent to the restricted setup. Restricted models receive previous actions as input, but often fail to recognize when they’re trapped in a cycle. This potentially encourages short-term decision-making, hindering their ability to analyze past actions and states. We suspect this issue is exacerbated by the requirement to return action numbers instead of names in the restricted setting. This implicit task progress representation might make it challenging for the model to consider the entire historical trajectory when choosing the next action. Consequently, the restricted version tends towards local optimizations, leading to frequent loop formation.

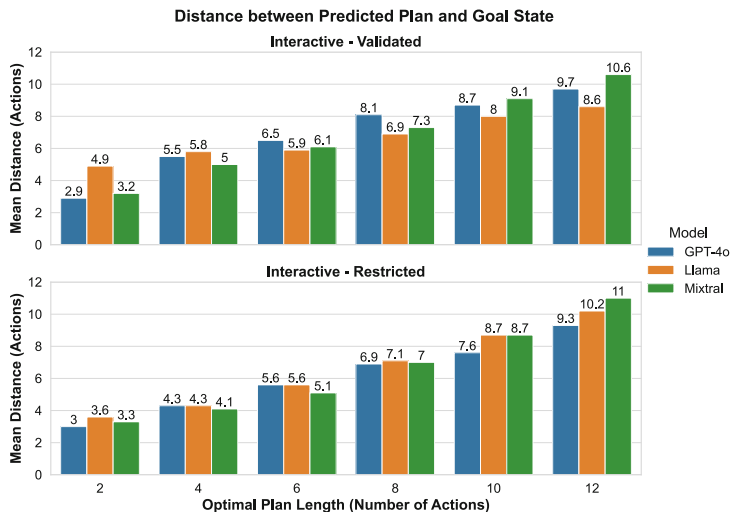


Fig. 5. The distance of the generated plan’s final state from the goal state is measured in terms of the minimum number of actions required to transition from the final state of the generated plan to the goal state. It was only calculated for plans that didn’t achieve the goal.

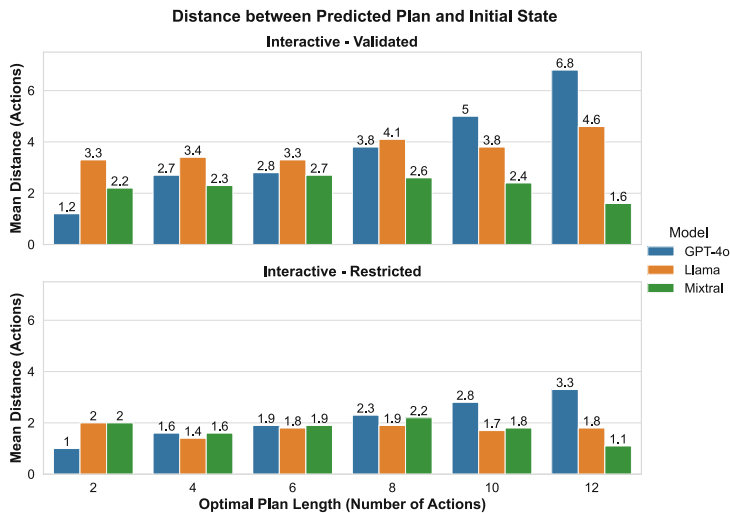


Fig. 6. The distance of the generated plan’s final state from the initial state is measured in terms of the minimum number of actions required to transition from the final state of the generated plan to the initial state. It was only calculated for plans that didn’t achieve the goal.

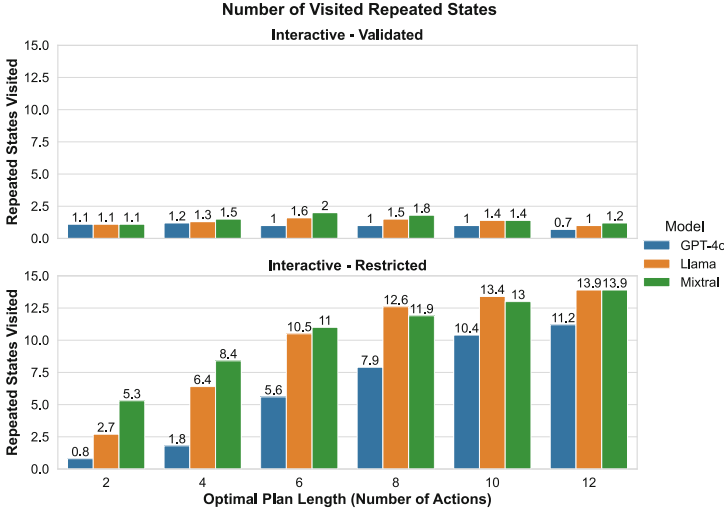


Fig. 7. We calculate the number of revisited states within a generated plan by summing up the occurrences of each state that appears more than once. This calculation is performed for all the plans generated.

6 Conclusion

Our results provide evidence that incorporating FSMs as a guiding structure enhances LLMs’ ability to solve planning problems. By interacting with the FSM and leveraging integrated validations, feedback loops, and restrictions, LLMs demonstrably generate plans with increased accuracy across tasks of varying complexity. Although the accuracy achieved is not yet ideal, the proposed method demonstrates a valuable tool for improving model inference in reasoning tasks, offering benefits independent of other prompt engineering techniques.

FSM integration was also compared to the CoT prompting technique in both preplanned and interactive validated schemes. CoT showed superior performance exclusively for the GPT-4o model, while FSM integration greatly increased accuracy for all models. This provides evidence that FSM integration is a more consistent solution than prompt engineering techniques that can behave unpredictably across different scenarios.

The proposed Interactive Validated and Restricted setups offer a crucial advantage by being inherently designed to handle invalid inputs, thus guaranteeing that all generated plans are valid. This is a substantial improvement over traditional approaches, where LLMs frequently generate invalid plans. We observed that incorporating validation encourages exploratory behavior in the models, leading to better performance on complex tasks across all three compared models. Conversely, while incorporating restrictions can enhance performance on simpler tasks, it can also cause the models to enter loops, resulting in poorer performance on more complex tasks. A significant downside of interactive

approaches is their increased computational cost. Results show that the number of tokens processed in these schemes is an order of magnitude higher than in preplanned setups, underscoring the necessity of prompt optimization in these scenarios.

Despite the simplicity of the experimental context, the findings open up new avenues for future research aimed at enhancing reasoning capabilities using logical structures that constrain the action space in problem-solving. The proposed method also introduces greater predictability and robustness into the system.

Our findings suggest promising avenues for future research. One direction involves developing approaches that can unify validation with feedback and restrictions within the LLM's decision-making process. This could lead to more reliable and robust models capable of both short-term and long-term reasoning.

References

1. Besta, M., et al.: Graph of thoughts: solving elaborate problems with large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, no. 16, pp. 17682–17690 (2024). <https://doi.org/10.1609/aaai.v38i16.29720>. <https://ojs.aaai.org/index.php/AAAI/article/view/29720>
2. Bubeck, S., et al.: Sparks of artificial general intelligence: early experiments with GPT-4 (2023)
3. Chen, M., et al.: Evaluating large language models trained on code (2021)
4. Ghallab, M., Nau, D., Traverso, P.: Automated Planning: Theory and Practice. The Morgan Kaufmann Series in Artificial Intelligence, Morgan Kaufmann, Amsterdam (2004). <http://www.sciencedirect.com/science/book/9781558608566>
5. Guan, L., Valmeekam, K., Sreedharan, S., Kambhampati, S.: Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems, vol. 36, pp. 79081–79094. Curran Associates, Inc. (2023)
6. Helmert, M.: The fast downward planning system. *J. Artif. Intell. Res.* **26**, 191–246 (2006). <https://doi.org/10.1613/jair.1705>
7. Hoffmann, J.: FF: the fast-forward planning system. *AI Mag.* **22**(3), 57 (2001). <https://doi.org/10.1609/aimag.v22i3.1572>. <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/1572>
8. Jiménez, S., De La Rosa, T., Fernández, S., Fernández, F., Borrajo, D.: A review of machine learning for automated planning. *Knowl. Eng. Rev.* **27**(4), 433–467 (2012). <https://doi.org/10.1017/S026988891200001X>
9. Jin, Z., et al.: Cladder: a benchmark to assess causal reasoning capabilities of language models (2023)
10. Raman, S.S., Cohen, V., Rosen, E., Idrees, I., Paulius, D., Tellex, S.: Planning with large language models via corrective re-prompting. In: NeurIPS 2022 Foundation Models for Decision Making Workshop (2022). <https://openreview.net/forum?id=cMDMRBe1TKs>
11. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach, 3 edn. Prentice Hall (2010)

12. Stechly, K., Marquez, M., Kambhampati, S.: GPT-4 doesn't know it's wrong: an analysis of iterative prompting for reasoning problems. In: NeurIPS 2023 Foundation Models for Decision Making Workshop (2023). <https://openreview.net/forum?id=PMtZjDYB68>
13. Suzgun, M., et al.: Challenging big-bench tasks and whether chain-of-thought can solve them (2022)
14. Valmeekam, K., Marquez, M., Olmo, A., Sreedharan, S., Kambhampati, S.: Plan-bench: an extensible benchmark for evaluating large language models on planning and reasoning about change. In: Thirty-Seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track (2023). <https://openreview.net/forum?id=YXogl4uQUO>
15. Valmeekam, K., Marquez, M., Sreedharan, S., Kambhampati, S.: On the planning abilities of large language models - a critical investigation. In: Thirty-Seventh Conference on Neural Information Processing Systems (2023). <https://openreview.net/forum?id=X6dEqXIsEW>
16. Vaswani, A., et al.: Attention is all you need. In: Guyon, I., et al. (eds.) *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc. (2017)
17. Wang, X., Wei, J., Schuurmans, D., Le, Q.V.: H. chi, sharan narang, aakanksha chowdhery, and denny zhou. self-consistency improves chain of thought reasoning in language models. In: *The Eleventh International Conference on Learning Representations*, vol. 1 (2023)
18. Wei, J., et al.: H. chi, tatsunori hashimoto, oriol vinyals, percy liang, jeff dean, and william fedus. emergent abilities of large language models. *Trans. Mach. Learn. Res.* **2022** (2022)
19. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837. Curran Associates, Inc. (2022)
20. Yao, S., et al.: Tree of thoughts: deliberate problem solving with large language models. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) *Advances in Neural Information Processing Systems*, vol. 36, pp. 11809–11822. Curran Associates, Inc. (2023)